

Machine learning for anomaly detection in finance

Jose H. Leon-Janampa

JHL Quantitative Analysis Ltd.

jhleonjy@yahoo.es

September 5, 2018

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Anomaly detection

- Anomaly detection is to identify unusual features that could form part of the tails of the expected distribution of some data but are not members of the true stable distribution of the relevant data.
- Definition of Hawkins D. (1980): "Outlier is an observation which deviates so much from the other observations as to arouse suspicion that it was generated by a different mechanism."
- A detection has a wide number of applications such as machine maintenance prediction, fraud detection for credit cards, outliers in non-categorized data, cyber security, surveillance.

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Benefits of anomaly detection 1/3

Operational risk facts are well known in financial market activities:

- Japanese broker placed orders for more than \$ 617 billions of stocks in a fat finger error (2014).
- Deutsche bank employee processed a \$6 billion trade instead of \$ 6 millions (2015).
- Pound value crash: Why did sterling plunge by 6 % in 2 minutes? (2016).
- BNP Paribas failed to book trades in Germany for a week (2015)
- A Clearinghouse Gained \$35 Billion Thanks to Deutsche Bank's Error (2018)

Motivations

Benefits of anomaly detection 2/3

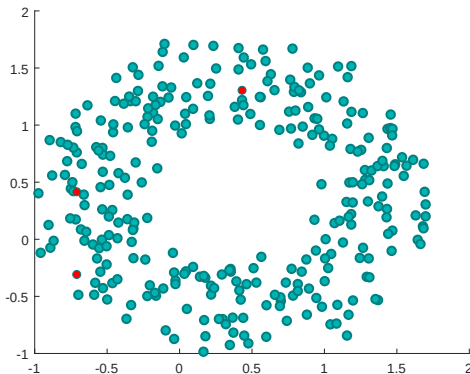


Figure: Anomalies in data

Benefits of anomaly detection 3/3

Anomaly detection

Identification of abnormal events (trades, payments, prices...)

Unexpected behavior of data has in the majority of cases a manual process of checking by middle office and back office. This makes the process pain-full and prone to errors. Normally covered by provisions inside operational risk.

Benefits of anomaly detection:

- Reduction of operational and reputational risk.
- Automation of manual validations and checks.
- Improving transaction costs accuracy, with good quality trades.
- Accurate pricing of derivatives.
- Less errors due to data in forecasting any economical random variable.

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Analysis based in different techniques:

- Deterministic validation rules.
- Empirical/Statistical methods.
- ML methods.

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- **Deterministic methods**
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Deterministic rules: static set of criteria (not necessarily based on past events or trades)

- **Example 1:** Trade date should not be after system entry date of the trade.
- **Example 2:** Counterparty is on a blacklist and should not be part of the trade.
- **Example 3:** Violation of any criteria well established in the market as normal/legal condition.

- Validations can be on...
 - Single data level (point anomaly checks)
 - Aggregated level, e.g. analyzing the sum of all trades with one counterparty in one day.
- Checks on different types of attributes of the data to analyse.
 - **Categorical:** Currency, portfolio, operator, counterparty, etc.
 - **Continuous:** Notional, net/gross amount, price.
 - **Binary:** Long/short position, good/bad credit etc.

1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- **Empirical/statistical methods**
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

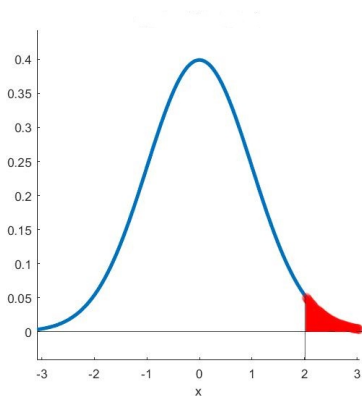
- The number of relevant features in data, for instance number of trades of a certain type is classified as similar in the past is below a certain predefined threshold so a new trade is abnormal.

- The number of relevant features in data, for instance number of trades of a certain type is classified as similar in the past is below a certain predefined threshold so a new trade is abnormal.
- Number of events or trades with similar categorical attributes have happened before.

- The number of relevant features in data, for instance number of trades of a certain type is classified as similar in the past is below a certain predefined threshold so a new trade is abnormal.
- Number of events or trades with similar categorical attributes have happened before.
- Identification of unlikely combinations in comparison to a historical pattern.

- **Example 1:** Trader X has never trade in a given portfolio, therefore a new trade from this trader is in that portfolio appears suspicious.
- **Example 2:** There has never been a trade in Chilean peso in a portfolio, a new trade as such is flagged as anomaly.
- **Example 3:** No historic trades in CDSs with a certain counterparty, so a CDS with this counterparty would be an anomaly.

Empirical/ statistical methods - continuous attributes



- Calculate the quantiles for the distributions of continuous features' attributes of similar features in the past, e.g. the 99 % quantile of the notional of all bonds in USD in a trading desk.
- In case the continuous features' attributes of the new feature we need to analyse exceeds the quantile as described above the feature is classified as anomalous.

1 Introduction

- Definitions
- Motivations

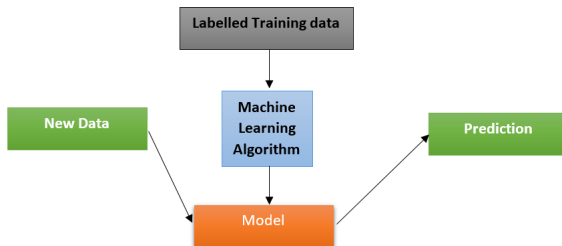
2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- **ML: Supervised Learning**
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

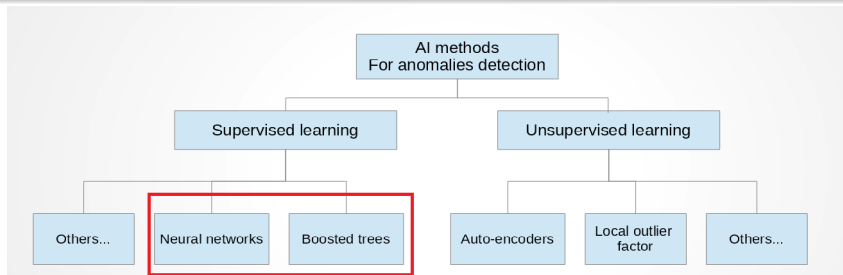
How to identify anomalies?: ML comes into play

- If we have historical labeled data, we can use that information to validate or classify new features and of course to train machine learning algorithms like neural networks, to identify new features, classify and regress.



Supervised Learning

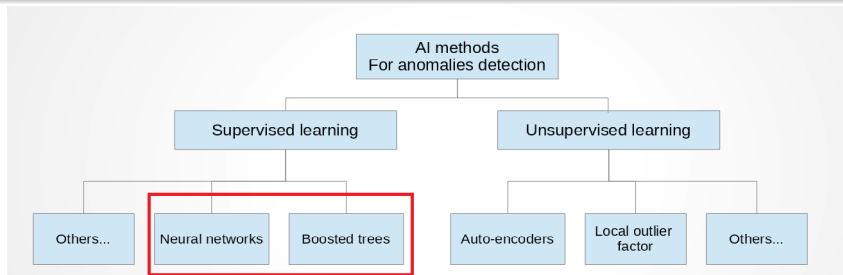
How to identify anomalies?: ML comes into play



- Supervised learning methods analyse past features and try to predict whether a new feature is likely to be confirmed as correct or to be modified or eliminated.

Supervised Learning

How to identify anomalies?: ML comes into play



- Supervised learning methods analyse past features and try to predict whether a new feature is likely to be confirmed as correct or to be modified or eliminated.
- Supervised learning with labeled training data can: confirm ("normal") features and modify ("abnormal") historical data.

1 Introduction

- Definitions
- Motivations

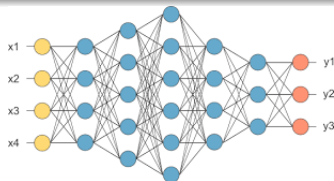
2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- **ML: Supervised Learning**
 - **Neural networks**
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Supervised Learning: Neural Networks

How to identify anomalies?: ML comes into play

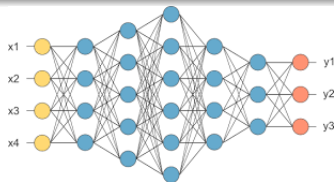


Algorithm

- Uses nodes connected to each other in different ways that mimic biological neurons with axons. And those connections given with certain geometry to produce appropriate information transfer.

Supervised Learning: Neural Networks

How to identify anomalies?: ML comes into play

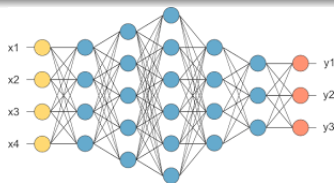


Algorithm

- Uses nodes connected to each other in different ways that mimic biological neurons with axons. And those connections given with certain geometry to produce appropriate information transfer.
- Learns tasks by inferring an approximate function $F(X)$ with outputs $Y = [y_1, \dots, y_n]$, with input features $X = [x_1, x_2, \dots, x_n]$ without a specific task-specific programming. Normally a feed-forward NN with back-propagation is enough to solve the problem.

Supervised Learning: Neural Networks

How to identify anomalies?: ML comes into play



Algorithm

- Uses nodes connected to each other in different ways that mimic biological neurons with axons. And those connections given with certain geometry to produce appropriate information transfer.
- Learns tasks by inferring an approximate function $F(X)$ with outputs $Y = [y_1, \dots, y_n]$, with input features $X = [x_1, x_2, \dots, x_n]$ without a specific task-specific programming. Normally a feed-forward NN with back-propagation is enough to solve the problem.
- The connectivity is driven by the weights that are adjusted during training.

How to identify anomalies?: ML comes into play

Algorithm input example:

- Training data: 50,000 historical features (13 % abnormal and 87 % normal features)
- Validation set: 2,000 features.

Algorithm	Recall	Precision	Accuracy	F1-Score
Neural network	36%	74 %	90 %	49 %

Table: Performance

Supervised Learning: Neural Networks

Recall : Probability that the feature is classified as abnormal in the validation set.

Precision: Probability that a feature classified as abnormal among the normal set.

Accuracy: Probability of correct classification.

F1-score: Harmonic mean of precision and recall.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (1)$$

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (2)$$

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total population}} \quad (3)$$

1 Introduction

- Definitions
- Motivations

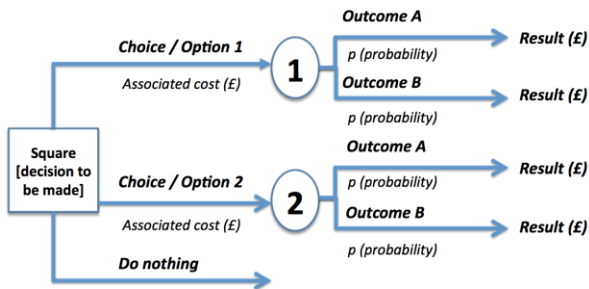
2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- **ML: Supervised Learning**
 - Neural networks
 - **Decision Trees and XGBoost**
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Supervised Learning: Decision Trees

Example of a decision tree



Tree base modeling: Introduction

Tree based learning algorithms are predictive models with high accuracy, stability and ease of interpretation. Unlike linear models, they map non-linear relationships quite well. They are adaptable at solving any kind of problem at hand (classification or regression).

Tree base modeling: Introduction

Tree based learning algorithms are predictive models with high accuracy, stability and ease of interpretation. Unlike linear models, they map non-linear relationships quite well. They are adaptable at solving any kind of problem at hand (classification or regression).

Decision trees, random forest, gradient boosting are some of the specific methods.

Tree base modeling: Introduction

Tree based learning algorithms are predictive models with high accuracy, stability and ease of interpretation. Unlike linear models, they map non-linear relationships quite well. They are adaptable at solving any kind of problem at hand (classification or regression).

Decision trees, random forest, gradient boosting are some of the specific methods.

Decision tree is based on the type of target variable we have. It can be of two types: Categorical Variable Decision Tree and Continuous Variable Decision Tree

Supervised learning: Decision trees

Advantages and disadvantages

Advantages

- **Easy to Understand:** It does not require any statistical knowledge.

Advantages and disadvantages

Advantages

- **Easy to Understand:** It does not require any statistical knowledge.
- **Useful in Data exploration:** Help us to identify most significant variables and relation between two or more variables. We can create new variables / features that has better power to predict target variable.

Advantages and disadvantages

Advantages

- **Easy to Understand:** It does not require any statistical knowledge.
- **Useful in Data exploration:** Help us to identify most significant variables and relation between two or more variables. We can create new variables / features that has better power to predict target variable.
- **Less data cleaning required:** It requires less data cleaning compared to some other modeling techniques. It is not influenced by outliers and missing values to a fair degree.

Supervised learning: Decision trees

Advantages and disadvantages

Advantages

- **Easy to Understand:** It does not require any statistical knowledge.
- **Useful in Data exploration:** Help us to identify most significant variables and relation between two or more variables. We can create new variables / features that has better power to predict target variable.
- **Less data cleaning required:** It requires less data cleaning compared to some other modeling techniques. It is not influenced by outliers and missing values to a fair degree.
- **Data type is not a constraint:** It can handle both numerical and categorical variables.

Supervised learning: Decision trees

Advantages and disadvantages

Advantages

- **Easy to Understand:** It does not require any statistical knowledge.
- **Useful in Data exploration:** Help us to identify most significant variables and relation between two or more variables. We can create new variables / features that has better power to predict target variable.
- **Less data cleaning required:** It requires less data cleaning compared to some other modeling techniques. It is not influenced by outliers and missing values to a fair degree.
- **Data type is not a constraint:** It can handle both numerical and categorical variables.
- **Non Parametric Method:** Decision trees have no assumptions about the space distribution and the classifier structure.

Disadvantages: Over fitting, Not fit for continuous variables.

Regression vs classification trees 1/2

- ① Regression trees are used when dependent variable is continuous.
Classification trees are used when dependent variable is categorical.

Regression vs classification trees 1/2

- 1 Regression trees are used when dependent variable is continuous. Classification trees are used when dependent variable is categorical.
- 2 In case of regression tree, the value obtained by terminal nodes in the training data is the mean response of observation falling in that region. Thus, if an unseen data observation falls in that region, we'll make its prediction with mean value.

Regression vs classification trees 1/2

- 1 Regression trees are used when dependent variable is continuous. Classification trees are used when dependent variable is categorical.
- 2 In case of regression tree, the value obtained by terminal nodes in the training data is the mean response of observation falling in that region. Thus, if an unseen data observation falls in that region, we'll make its prediction with mean value.
- 3 In case of classification tree, the value (class) obtained by terminal node in the training data is the mode of observations falling in that region.

Supervised Learning: Decision trees

Regression vs classification trees 1/2

- 1 Regression trees are used when dependent variable is continuous. Classification trees are used when dependent variable is categorical.
- 2 In case of regression tree, the value obtained by terminal nodes in the training data is the mean response of observation falling in that region. Thus, if an unseen data observation falls in that region, we'll make its prediction with mean value.
- 3 In case of classification tree, the value (class) obtained by terminal node in the training data is the mode of observations falling in that region.
- 4 Both the trees divide the predictor space (independent variables) into distinct and non-overlapping regions.

Regression vs classification trees 1/2

- ➊ Regression trees are used when dependent variable is continuous. Classification trees are used when dependent variable is categorical.
- ➋ In case of regression tree, the value obtained by terminal nodes in the training data is the mean response of observation falling in that region. Thus, if an unseen data observation falls in that region, we'll make its prediction with mean value.
- ➌ In case of classification tree, the value (class) obtained by terminal node in the training data is the mode of observations falling in that region.
- ➍ Both the trees divide the predictor space (independent variables) into distinct and non-overlapping regions.
- ➎ Both the trees follow a top-down greedy approach known as recursive binary splitting.

Regression vs classification trees 2/2

- 6 The splitting process is continued until a user defined stopping criteria is reached.

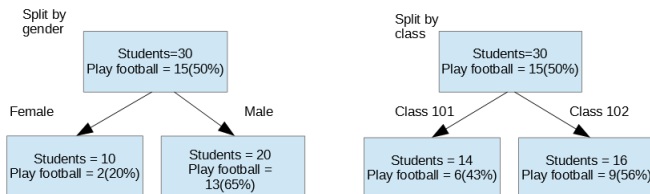
Regression vs classification trees 2/2

- ⑥ The splitting process is continued until a user defined stopping criteria is reached.
- ⑦ In both the cases, the splitting process results in fully grown trees until the stopping criteria is reached. But, the fully grown tree is likely to overfit data, leading to poor accuracy on unseen data. This bring "pruning".

Supervised Learning: Decision trees

Tree base modeling: How decision trees work

- Decision tree is a type of supervised learning algorithm (having a pre-defined target variable) that is mostly used in classification problems. It works for both categorical and continuous input and output variables.
- The sample is split into two or more homogeneous sets (or sub-samples) based on most significant splitter/ differentiator in input variables.



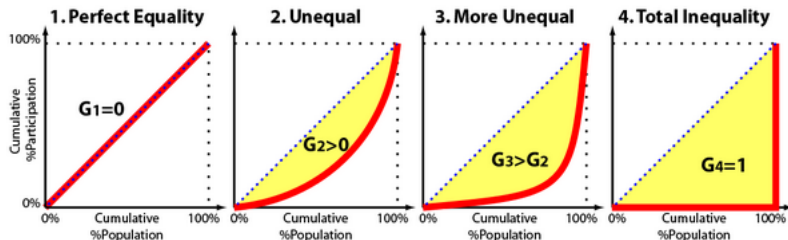
How a tree decide where to split ?

- 1 Decision trees use multiple algorithms to decide to split a node in two or more sub-nodes. The creation of sub-nodes increases the homogeneity of resultant sub-nodes. In other words, we can say that purity of the node increases with respect to the target variable. Decision tree splits the nodes on all available variables and then selects the split which results in most homogeneous sub-nodes.
- 2 The algorithm selection is also based on type of target variables. Among the most common scoring metrics used for splitting are: Gini index, Chi square, Information gain, reduction in variance.

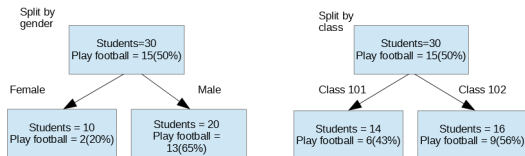
Supervised Learning: Decision trees

Gini index in trees: Is a metric of statistical dispersion or inequality, $G = 0$ perfect equality. In the case of a decision tree we can apply it as follows.

- Calculate Gini for sub-nodes, using formula sum of square of probability for success and failure ($p^2 + q^2$).
- Calculate Gini for split, using weighted Gini score of each node of that split.



Supervised Learning: Decision trees



- 1 Split by gender: Calculate, Gini for sub-node
 $Female = (0.2) * (0.2) + (0.8) * (0.8) = 0.68$
- 2 Gini for sub-node $Male = (0.65) * (0.65) + (0.35) * (0.35) = 0.55$
- 3 Calculate weighted Gini for Split
 $Gender = (10/30) * 0.68 + (20/30) * 0.55 = 0.59$
- 4 In a similar way you can calculate weighted Gini for Split Class = 0.51

you can see that Gini score for Split on Gender is higher than Split on Class, hence, the node split will take place on Gender.

How can we avoid over-fitting in decision trees? It can be done in 2 ways:

- ① Setting constraints on tree size.
- ② Tree pruning

How can we avoid over-fitting in decision trees? It can be done in 2 ways:

- ① Setting constraints on tree size.
- ② Tree pruning

Constraints on tree size

- **Minimum number of samples** required in a node for splitting. High values prevent a model from learning relations which might be highly specific to the particular sample.

Supervised Learning: Decision trees

How can we avoid over-fitting in decision trees? It can be done in 2 ways:

- ① Setting constraints on tree size.
- ② Tree pruning

Constraints on tree size

- **Minimum number of samples** required in a node for splitting. High values prevent a model from learning relations which might be highly specific to the particular sample.
- Limit the **minimum leaf samples** (or observations) required in a terminal node or leaf. Used to control over-fitting similar to min samples split.

How can we avoid over-fitting in decision trees? It can be done in 2 ways:

- ① Setting constraints on tree size.
- ② Tree pruning

Constraints on tree size

- **Minimum number of samples** required in a node for splitting. High values prevent a model from learning relations which might be highly specific to the particular sample.
- Limit the **minimum leaf samples** (or observations) required in a terminal node or leaf. Used to control over-fitting similar to min samples split.
- The **maximum depth** of a tree. Used to control over-fitting as higher depth will allow model to learn relations very specific to a particular sample.

How can we avoid over-fitting in decision trees?

- The **maximum number of terminal nodes**, similar to max-depth.

How can we avoid over-fitting in decision trees?

- The **maximum number of terminal nodes**, similar to max-depth.
- The **number of features to consider while searching for a best split**. As a thumb-rule, square root of the total number of features works great but we should check upto 30-40 % of the total number of features. Higher values can lead to over-fitting but depends on case to case.

How can we avoid over-fitting in decision trees?

- The **maximum number of terminal nodes**, similar to max-depth.
- The **number of features to consider while searching for a best split**. As a thumb-rule, square root of the total number of features works great but we should check upto 30-40 % of the total number of features. Higher values can lead to over-fitting but depends on case to case.

Tree pruning: First make the decision tree to a large depth. Then we start at the bottom and start removing leaves which are giving us negative returns when compared from the top. Suppose a split is giving us a gain of say -10 (loss of 10) and then the next split on that gives us a gain of 20. A simple decision tree will stop at step 1 but in pruning, we will see that the overall gain is +10 and keep both leaves.

Supervised Learning: Improving decision trees

Ensemble methods and bagging

Ensemble methods and bagging

- Ensemble methods involve group of predictive models to achieve a better accuracy and model stability.

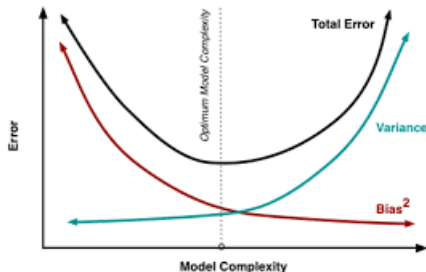
Ensemble methods and bagging

- Ensemble methods involve group of predictive models to achieve a better accuracy and model stability.
- Address the bias - variance problem.

Supervised Learning: Improving decision trees

Ensemble methods and bagging

- Ensemble methods involve group of predictive models to achieve a better accuracy and model stability.
- Address the bias - variance problem.
- **Bagging** is a technique used to reduce the variance of our predictions by combining the result of multiple classifiers modeled on different sub-samples of the same data set. Step 1. Multiple datasets, Step2. multiple classifiers, Step 3. combination. Variance



Supervised Learning: What is Boosting ? How does it work?

The term “Boosting” refers to a family of algorithms which converts weak learner to strong learners.

Supervised Learning: What is Boosting ? How does it work?

The term “Boosting” refers to a family of algorithms which converts weak learner to strong learners.

Let us look at the SPAM example: An email is “spam” and “not spam” emails using following criteria. If:

- 1 Email has only one image file (promotional image), It's a SPAM.
- 2 Email has only link(s), It's a SPAM.
- 3 Email body consist of sentence like “You won a prize money of \$ xxxxxx”, It's a SPAM
- 4 Email from our official domain “BBC.co.uk.com” , Not a SPAM
- 5 Email from known source, Not a SPAM

Supervised Learning: What is Boosting ? How does it work?

The term “Boosting” refers to a family of algorithms which converts weak learner to strong learners.

Let us look at the SPAM example: An email is “spam” and “not spam” emails using following criteria. If:

- 1 Email has only one image file (promotional image), It's a SPAM.
- 2 Email has only link(s), It's a SPAM.
- 3 Email body consist of sentence like “You won a prize money of \$ xxxxxx”, It's a SPAM
- 4 Email from our official domain “BBC.co.uk.com” , Not a SPAM
- 5 Email from known source, Not a SPAM

Individually, these rules are not powerful enough to classify an email into “spam” or “not spam”. Therefore, these rules are called as weak learner. To convert weak learner to strong learner use (1) average/ weighted average (2) higher vote prediction.

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Apply base learning (ML) algorithms (a decision criteria) with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Apply base learning (ML) algorithms (a decision criteria) with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

For choosing the right distribution, here are the following steps:

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Apply base learning (ML) algorithms (a decision criteria) with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

For choosing the right distribution, here are the following steps:

- Step 1: The base learner takes all the distributions and assign equal weight or attention to each observation.

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Apply base learning (ML) algorithms (a decision criteria) with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

For choosing the right distribution, here are the following steps:

- Step 1: The base learner takes all the distributions and assign equal weight or attention to each observation.
- Step 2: If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.

Supervised Learning: What is Boosting ? How does it work?

How boosting identify weak rules?.

Apply base learning (ML) algorithms (a decision criteria) with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

For choosing the right distribution, here are the following steps:

- Step 1: The base learner takes all the distributions and assign equal weight or attention to each observation.
- Step 2: If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.
- Step 3: Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

- XGBoost is also known as 'regularized boosting' technique.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

- XGBoost is also known as 'regularized boosting' technique.
- XGBoost implements parallel processing even it is a sequential process.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

- XGBoost is also known as 'regularized boosting' technique.
- XGBoost implements parallel processing even it is a sequential process.
- XGBoost allow users to define custom optimization objectives and evaluation criteria.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

- XGBoost is also known as 'regularized boosting' technique.
- XGBoost implements parallel processing even it is a sequential process.
- XGBoost allow users to define custom optimization objectives and evaluation criteria.
- XGBoost has an in-built routine to handle missing values.

XGBoost one of the best methods so far

What if we add to a decision tree, bagging, boosting, regularization, pruning and other nice efficient properties?, then we get XGBoost. It has the following good properties.

- XGBoost is also known as 'regularized boosting' technique.
- XGBoost implements parallel processing even it is a sequential process.
- XGBoost allow users to define custom optimization objectives and evaluation criteria.
- XGBoost has an in-built routine to handle missing values.
- XGBoost make splits upto the max-depth specified and then start pruning the tree backwards and remove splits beyond which there is no positive gain.

How to identify anomalies using Boosted trees?

- 1 Build decision tree No.1 in order to predict the anomaly ($y=0/1$)

How to identify anomalies using Boosted trees?

- 1 Build decision tree No.1 in order to predict the anomaly ($y=0/1$)
- 2 Build decision tree No.2 by overweighting samples that are wrongly classified by tree No.1 and underweight samples which are correctly classified by tree No.1

How to identify anomalies using Boosted trees?

- 1 Build decision tree No.1 in order to predict the anomaly ($y=0/1$)
- 2 Build decision tree No.2 by overweighting samples that are wrongly classified by tree No.1 and underweight samples which are correctly classified by tree No.1
- 3 ...
- 4 Build decision tree No.n to improve the residual error sequentially.

$$P_{\text{Boosted tree}} = \sum_i w_i \text{Tree}_i(x) \quad (4)$$

A typical example is to have 200 or 300 trees with a depth of 5 or 6.

Results of an XGBoost application

Algorithm input example:

- Training data: 50,000 historical features (13 % abnormal and 87 % normal features)
- Validation set: 2,000 features.

Algorithm	Recall	Precision	Accuracy	F1-Score
Neural network	36%	74 %	90 %	49 %
Boosted trees	60 %	98 %	96 %	71 %

Table: Performance

1 Introduction

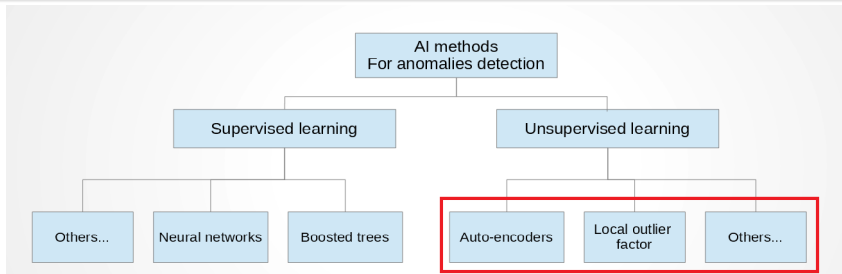
- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- **ML: Unsupervised learning**
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

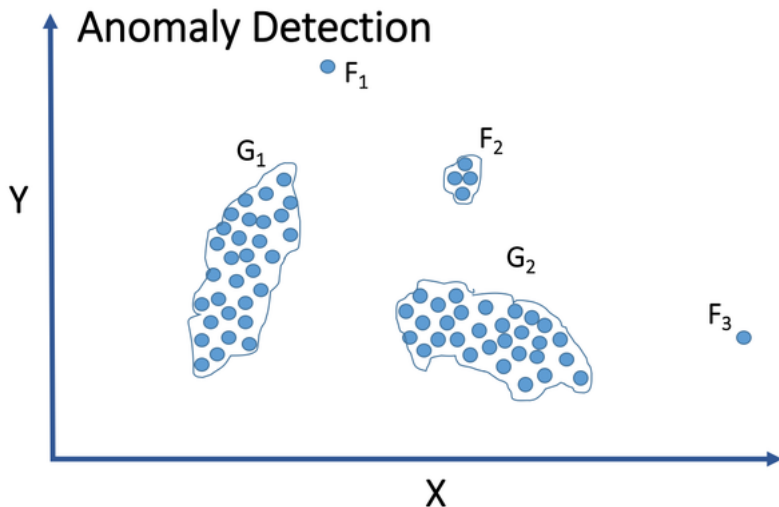
How to identify anomalies?



- If we do not have labeled data, we need to use unsupervised learning methods in order to detect unusual patterns in the input data.

Unsupervised Learning

How to identify anomalies?



1 Introduction

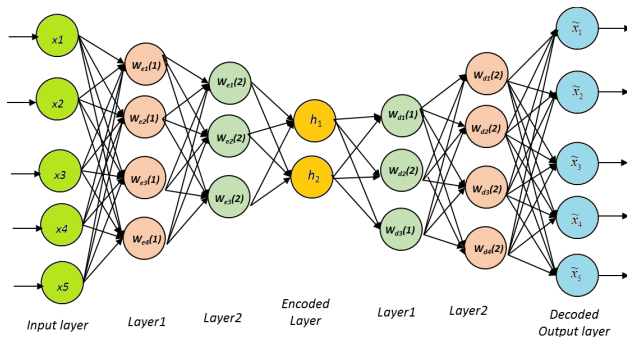
- Definitions
- Motivations

2 Anomaly detection techniques

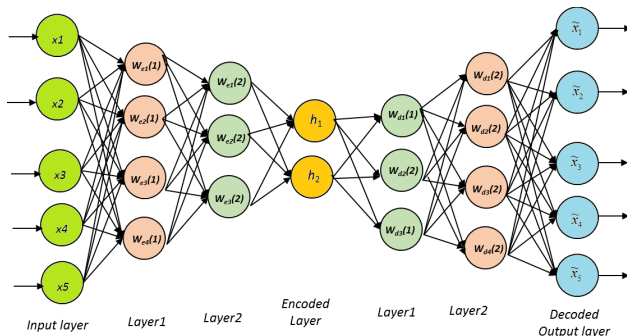
- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
- Other methods

3 Conclusions

Unsupervised Learning: Autoencoder

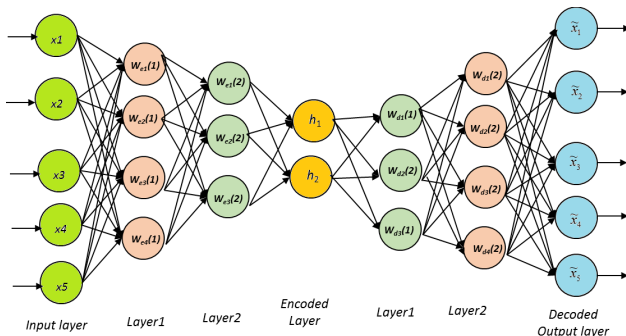


Unsupervised Learning: Autoencoder



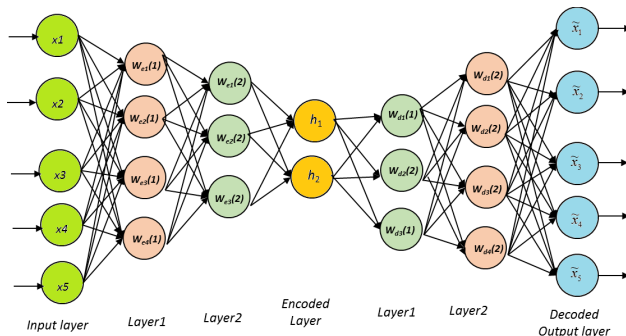
- An **autoencoder** is an artificial neural network which trains the network with a bottleneck structure which objective is to replicate the input data.

Unsupervised Learning: Autoencoder



- An **autoencoder** is an artificial neural network which trains the network with a bottleneck structure which objective is to replicate the input data.
- Goal: To extract key features of the input data and ignoring secondary effects.

Unsupervised Learning: Autoencoder



- An **autoencoder** is an artificial neural network which trains the network with a bottleneck structure which objective is to replicate the input data.
- Goal: To extract key features of the input data and ignoring secondary effects.
- Similar to other dim. reduction algos but this time capturing non-linear dependencies.

Unsupervised Learning: Autoencoder

Autoencoders for anomaly detection:

Autoencoders for anomaly detection:

- Step 1: Measure the reconstruction error (e.g. mean squared error, etc.)

Autoencoders for anomaly detection:

- Step 1: Measure the reconstruction error (e.g. mean squared error, etc.)
- Step 2: Choose a threshold for reconstruction error for outlier detection.

Autoencoders for anomaly detection:

- Step 1: Measure the reconstruction error (e.g. mean squared error, etc.)
- Step 2: Choose a threshold for reconstruction error for outlier detection.
- Step 3: Compute the reconstructed output y given a new input sample x_{new} using the trained autoencoder.

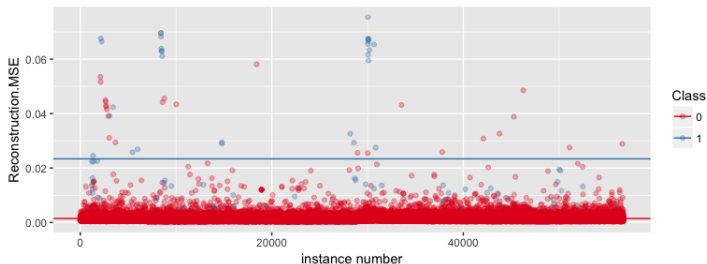
Autoencoders for anomaly detection:

- Step 1: Measure the reconstruction error (e.g. mean squared error, etc.)
- Step 2: Choose a threshold for reconstruction error for outlier detection.
- Step 3: Compute the reconstructed output y given a new input sample x_{new} using the trained autoencoder.
- Step 4: If the reconstruction error between x and y is above threshold then x_{new} is an outlier.

Unsupervised Learning: Autoencoder

Autoencoders for anomaly detection:

- Step 1: Measure the reconstruction error (e.g. mean squared error, etc.)
- Step 2: Choose a threshold for reconstruction error for outlier detection.
- Step 3: Compute the reconstructed output y given a new input sample x_{new} using the trained autoencoder.
- Step 4: If the reconstruction error between x and y is above threshold then x_{new} is an outlier.



1 Introduction

- Definitions
- Motivations

2 Anomaly detection techniques

- Deterministic methods
- Empirical/statistical methods
- ML: Supervised Learning
 - Neural networks
 - Decision Trees and XGBoost
- ML: Unsupervised learning
 - Autoencoder
 - Local Outlier Factor (LOF)
 - Other methods

3 Conclusions

Basic assumption:

- Normal data objects have a dense neighborhood which is similar to the density of their neighbors.

Basic assumption:

- Normal data objects have a dense neighborhood which is similar to the density of their neighbors.
- Outliers are far apart from their neighbors and have much less dense neighborhood in comparison to the density of their neighbors.

Basic assumption:

- Normal data objects have a dense neighborhood which is similar to the density of their neighbors.
- Outliers are far apart from their neighbors and have much less dense neighborhood in comparison to the density of their neighbors.

Definition: k-distance of an object

p: k-distance(p) of object p is defined as distance $d(p, o)$ between p and an object $o \in D$ such that:

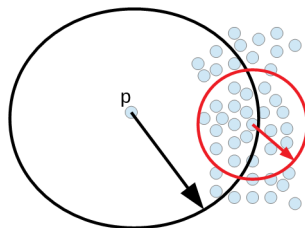
Basic assumption:

- Normal data objects have a dense neighborhood which is similar to the density of their neighbors.
- Outliers are far apart from their neighbors and have much less dense neighborhood in comparison to the density of their neighbors.

Definition: k -distance of an object

p: k -distance(p) of object p is defined as distance $d(p, o)$ between p and an object $o \in D$ such that:

- for at least k objects o' it holds that $d(p, o') \leq d(p, o)$
- for at most $k - 1$ object o' it holds that $d(p, o') < d(p, o)$



- **Definition: k-distance neighborhood of an object p**

k-distance neighborhood of object p contains every object whose distance from p is not greater than the k -distance(p), i.e.

$N_{kNN-distance}(p) = \{q \in D - p | d(p, q) \leq k - distance(p)\}$. These objects q are called the k-nearest neighbors of p .

- **Definition: k-distance neighborhood of an object p**

k-distance neighborhood of object p contains every object whose distance from p is not greater than the k -distance(p), i.e.

$N_{kNN-distance}(p) = \{q \in D - p \mid d(p, q) \leq k - distance(p)\}$. These objects q are called the k-nearest neighbors of p .

- **kNN-local density of an object p** The local density of object p is defined as:

$$\text{local density kNN}(p) = \frac{1}{\frac{\sum_{o \in N_{k-distance}(p)} \max[d(p, o); k - dist(o)]}{|N_{kNN-dist}(p)|}} \quad (5)$$

Inverse of the average of the k-nearest neighbors of p .

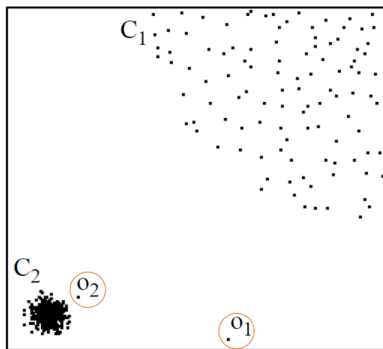
- **Definition: Local Outlier Factor (LOF)** is defined as the average of the ratio of the density of the kNN-distance neighborhood of p and the densities of the neighborhood of the k-nearest neighbors of p .

$$LOF_k(p) = \left(\frac{\sum_{o \in N_{kNN-distance}(p)} \frac{\text{local density}_{kNN}(o)}{\text{local density}_{kNN}(p)}}{|N_{kNN-dist}(p)|} \right) \quad (6)$$

- $LOF \approx 1$ for normal features in dense neighborhoods.
- LOF is significantly above 1 for outliers.

Local Outlier Factor

- The local Outlier Factor method is able to detect o_1 as well as o_2 as outliers in the figure below.
- The method can be extended to provide an outlier 'score' in the range of $[0, 1]$ that would be directly interpretable as a probability of the data feature for being an outlier.



There are many other methods for anomaly detection:

- Hybrid Forecasting of the time series considered using an Auto-Arima approach (component A). See Hyndman 2017 and Hyndman Khandakar 2008, then we consider an Autoregressive Neural Network in order to analyze the residuals of the series (component B). Zhang, P. (2012, April).
- LSTM models for anomaly detection.

Conclusions

- Anomaly detection is a very important topic in the industry in particular in finance. Because it can optimize many processes in different subareas in the front back and middle offices. It can help decreasing operational and reputational risk. Increase risk assesment, trade quality and profitability.
- Anomaly detection can reduce manual validation checks of trade activities significantly.
- Deterministic rules and standard statistical methods are an important building blocks of a trade anomaly detection application.
- Powerful supervised learning algorithms can be applied for trade anomaly detection if labeled past data is available.
- In the absence of labeled past data, unsupervised learning algorithms can be used in order to detect unusual patterns in the financial related activities.
- Anomaly detection will become a standard feature of the all areas in the financial industry.

The End

Appendix 1. A decision tree example

Given the following time series, the task is to predict if John will play football

Predict if John play football

Day	Outlook	Humidity	Wind	Play
d1	Sunny	High	Weak	No
d2	Sunny	High	Strong	No
d3	Overcast	High	Weak	Yes
d4	Rain	High	Weak	Yes
d5	Rain	Normal	Weak	Yes
d6	Rain	Normal	Strong	No
d7	Overcast	Normal	Strong	Yes
d8	Sunny	High	Weak	No
d9	Sunny	Normal	Weak	Yes
d10	Rain	Normal	Weak	Yes
d11	Sunny	Normal	Strong	Yes
d12	Overcast	High	Strong	Yes
d13	Overcast	Normal	Weak	Yes
d14	Rain	High	Strong	No

New data:

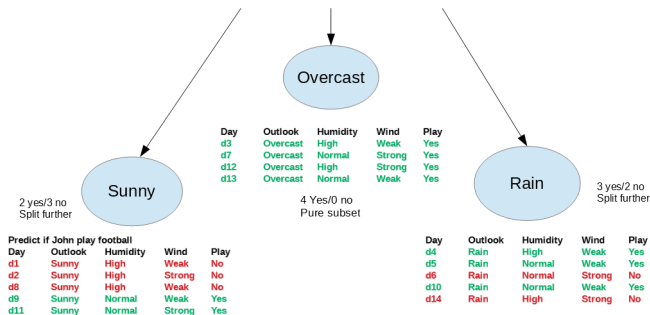
d15	Rain	High	Weak	????
-----	------	------	------	------

Appendix 1. A decision tree example

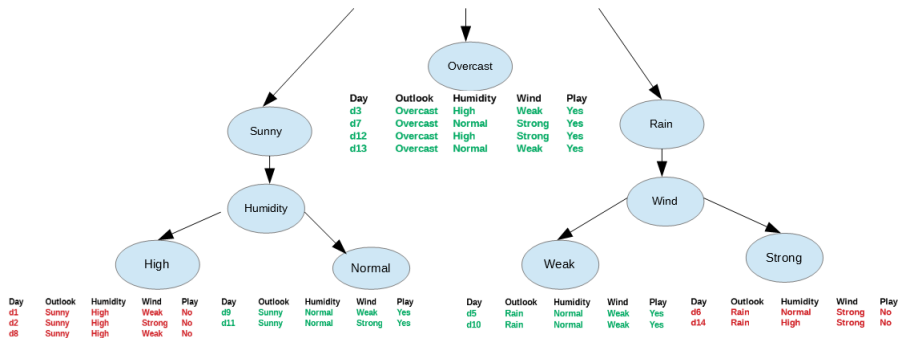
Predict if John will play football:

The idea in a decision tree is to understand when John will play football, based on the use of attributes or factors present in the data. In this case these factors are 'outlook', 'humidity', 'wind'. Using these attributes, the decision tree is going to make an inference if John will play football conditioned to the realization of any of these factors. The algorithm is going to use these attributes to split the data into subsets or 'distributions'. For instance if we look at the weather; it could be 'sunny', 'overcast' or 'rainy', these are 3 subsets, with some examples in each subset. If one state is pure like John plays when it is raining then the algorithm stops, otherwise we continue by looking to other attributes and split further. When a new data comes, we will look at which subset this sample falls into.

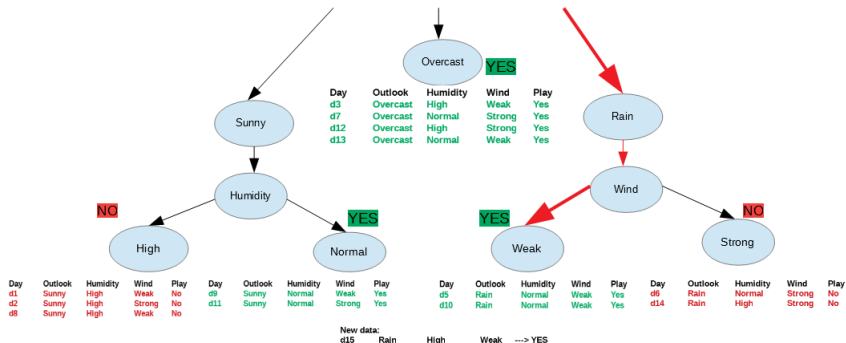
Appendix 1. A decision tree example



Appendix 1. A decision tree example



Appendix 1. A decision tree example



Appendix 2. Supervised Learning: XGBoost

How does XGBoost learn?

- Model: assume we have K trees

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad \text{where } f_k \in \mathcal{F} \quad (7)$$

- Objective

$$Obj = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (8)$$

- Optimizing training loss encourages fitting well data, which is hopefully close to the underlying distribution.
- Optimizing regularization encourages simple models, smaller variance in future predictions, making prediction stable.

Appendix 2. Supervised Learning: XGBoost

Define the complexity of the tree

- In terms of the leaf weight of the tree w , and the structure of the tree q

$$f_i(x) = w_{q(x)}, \quad \text{where } w \in R^T, q: \mathbf{R}^d \rightarrow 1, 2, \dots, T \quad (9)$$

- The regularization term

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (10)$$

T is the number of leaves.

Appendix 2. Supervised Learning: XGBoost

How can we learn tree ensembles

- Use additive training (Boosting)
- Start from a constant prediction, add a new function each time

$$\begin{aligned}\hat{y}_i^{(0)} &= 0 \\ \hat{y}_i^{(1)} &= \hat{y}_i^{(0)} + f_1(x_i) \\ \hat{y}_i^{(2)} &= f_1(x_i) + f_2(x_i) = \hat{y}_i^{(1)} + f_2(x_i) \\ &\dots \\ \hat{y}_i^{(t)} &= \sum_{k=1}^t f_k(x_i) = \hat{y}_i^{(t-1)} + f_t(x_i)\end{aligned}\tag{11}$$

$\hat{y}_i^{(t)}$ is the model at training round t . $\hat{y}_i^{(t-1)}$ is the model at the previous round. $f_t(x_i)$ the new function.